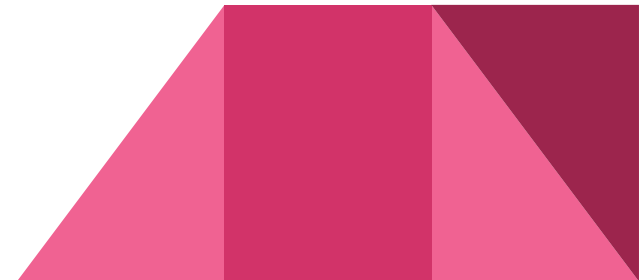


Haskell și Metode Formale

Laborator 2

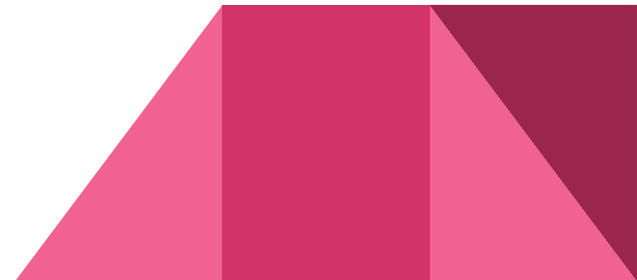
Ce este Haskell?

- Limbaj funcțional pur
- Tipare statice și puternice
- Evaluare leneșă
- Fundamentat pe lambda calcul



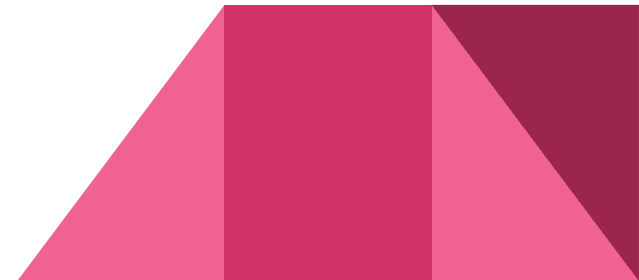
Haskell și Metodele Formale

- Funcții pure $\rightarrow$ raționament matematic simplu
- Tipurile acționează ca specificații formale
- Concepte din teoria categoriilor
- Transparență referențială



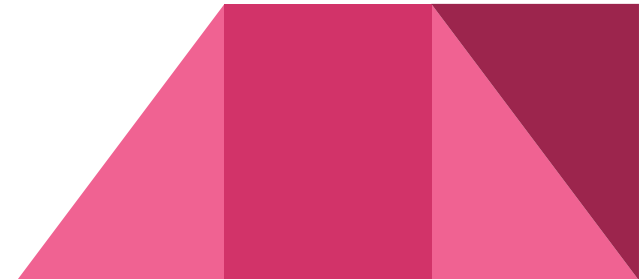
Specificarea formală în Haskell

- Tipuri ca specificații
- Legi funcționale
- Proprietăți matematice (ex.: $\text{reverse} (\text{reverse } xs) = xs$)



QuickCheck

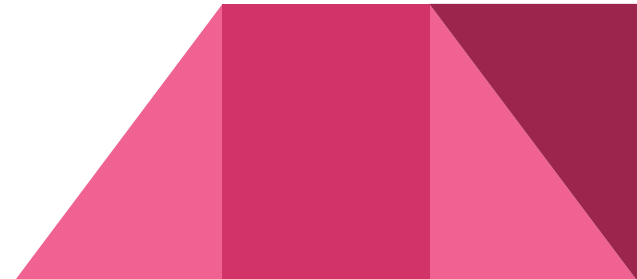
- Bibliotecă pentru Property-Based Testing
- Teste generate automat
- Caută contraexemple



Exemplu QuickCheck

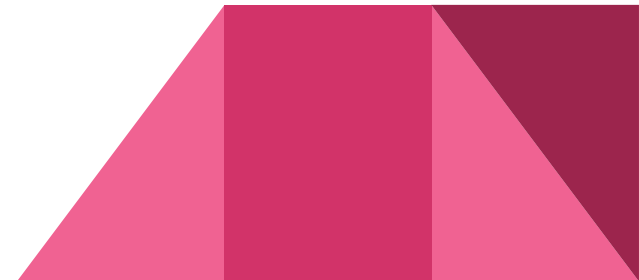
```
prop_reverse :: [Int] -> Bool
```

```
prop_reverse xs = reverse (reverse xs) == xs
```



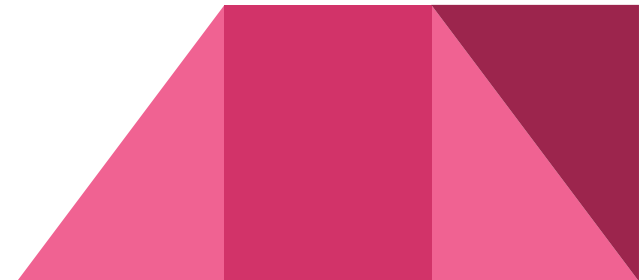
Definirea unui limbaj miniatural

- Expresii aritmetice
- Tipuri de date recursive
- Evaluare prin interpretare



Exemplu: Limbaj aritmetic

data Expr = Const Int | Add Expr Expr | Mul Expr
Expr



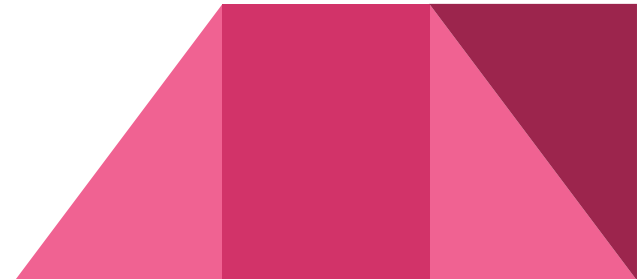
Interpretarea expresiilor

$\text{eval} :: \text{Expr} \rightarrow \text{Int}$

$\text{eval} (\text{Const } n) = n$

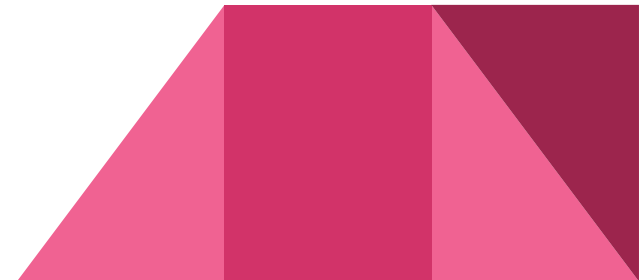
$\text{eval} (\text{Add } e1 \ e2) = \text{eval } e1 + \text{eval } e2$

$\text{eval} (\text{Mul } e1 \ e2) = \text{eval } e1 * \text{eval } e2$



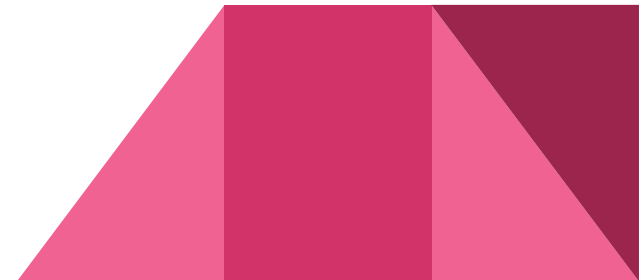
Proprietăți formale pentru limbaj

- Comutativitate: $\text{Add } e1 \ e2 \equiv \text{Add } e2 \ e1$
- Asociativitate
- Legi de simplificare



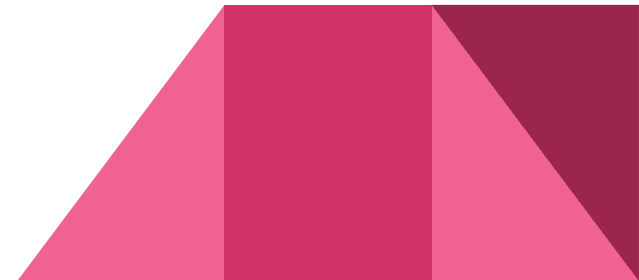
QuickCheck pentru limbaj

- `prop_add_comm e1 e2 = eval (Add e1 e2) == eval (Add e2 e1)`



De la specificație la implementare

- Definirea tipurilor
- Definirea semanticii formale
- Implementare în Haskell
- Verificare prin QuickCheck



Avantaje Haskell

- Modelare matematică
- Limbaje miniaturale ușor de definit
- Testare automatizată formală

